

Oblig 8

ITF20006-1 25V Algoritmer og datastrukturer

Andreas Isaksen

21. mars 2025
Halden



Innhold

1 Teori	1
1.1 Oppgave D	1

Teori

1.1 Oppgave D

Hvilken orden, angitt med O-notasjon, har tidsforbruket til metoden `LagHeapOrdning()` fra forrige oppgave, dersom treet inneholder n noder og:

- a) Treet er komplett, dvs. at alle nivåer, muligens bortsett fra det dypeste nivået, er helt fylt opp med noder?
- Metoden `lagHeapOrdning()` går rekursivt gjennom alle nodene i treet og kaller på `reparer()` i hver node.
 - `reparer()` metoden vil gjennom iterasjonen behandle alle node verdier og 'boble' de lavere verdiene oppover i treet.
 - i et **komplett binært tre** vil høyden være $h = \log_2(n)$ og i verste tilfelle vil `reparer()` metoden bruke $O(\log n)$ tid.
 - I hver n -te node brukes $O(\log n)$ tid på å reparere 'Heap ordningen'.
 - Dette gir oss et totalt tidsforbruk på:

$$\underline{\underline{O(n \cdot \log n)}}$$

- b) Hver node (bortsett fra én) har nøyaktig ett subtre (som ikke er tomt), og den siste noden har ingen ikke-tomme subtrær?
- Et slikt tre vil være svært ubalansert/ensidig og ligne mer på en *lenket liste* (med unntak av den siste noden).
 - Antall noder vil fortsatt være n .
 - Metoden `lagHeapOrdning()` vil fortsatt gå rekursivt gjennom treet og kalle på `reparer()` metoden for hver node, men grunnet treet's 'fasong', så blir dette mer som å iterere gjennom en *lenket liste*.
 - Dermed må `reparer()` metoden i verste fall gå hele veien i bladet for en node ($O(n)$).
 - Dette gir oss et totalt tidsforbruk på:

$$\underline{\underline{O(n \cdot n) = O(n^2)}}$$